Devups

Tome III — L'interface

Documentation du framework

Deuxième édition — novembre 2026 · Devups 7.0.0

Établie à partir du code du dépôt **spacekola/devups** (github.com/yuricoco/devups-package), version 7.0.0, et de sa suite de tests.

Sommaire

Chapitre 1 — Les annotations de route 4

1.1 Le lecteur d'annotations 4
1.2 Les annotations disponibles 4
1.3 Les segments de chemin 4
1.4 La barre oblique initiale 5

Chapitre 2 — Les trois styles d'API 6

2.1 Style générique — `<action>/<entité>` 6
2.2 Style REST dédié — un `FrontController` par entité 7
2.3 Style libre — `<action>Serve()` 9
2.4 Lequel choisir 9

Chapitre 3 — Les contrôleurs 11

3.1 Les quatre familles 11
3.2 Le contrôleur d'administration 11
3.3 `CrudTrait` — le CRUD en six méthodes 11
3.4 Le remplissage d'une entité depuis une requête 12
3.5 Les écouteurs d'événements 12
3.6 Les contrôleurs de module 13
3.7 La dégradation gracieuse 13

Chapitre 4 — Le moteur de listes 14

4.1 Le contrat déclaratif 14
4.2 Les quatre formes d'une colonne 14
4.3 Les autres clés d'une entrée 15
4.4 Les propriétés de comportement 15
4.5 Les méthodes utiles 16
4.6 Le vocabulaire de la requête 16
4.7 Les opérateurs de filtre 17
4.8 La réponse de `lazyloading` 17

Chapitre 5 — Le moteur de formulaires 19

5.1 Les trois classes 19
5.2 Le contrat déclaratif 19
5.3 Les types de champ 20
5.4 Les clés d'un champ 20
5.5 Les aides à la construction des options 20
5.6 Les méthodes de `FormManager` 21
5.7 L'API « à la main » 21
5.8 Un champ traduit 21

Chapitre 6 — Les vues 22

6.1 Blade 22

6.2 `Genesis` — le rendu ... 22
6.3 Les fonctions globales utiles ... 22
6.4 Le JavaScript du socle ... 23

Chapitre 7 — Les langues de l'interface ... 24
7.1 Le code ISO à deux lettres, partout ... 24
7.2 Les fichiers de traduction ... 24
7.3 Les fonctions ... 24
7.4 La collecte des clés manquantes ... 25
7.5 Les commandes ... 25

Chapitre 8 — La sécurité ... 26
8.1 Les deux systèmes d'identité ... 26
8.2 Le JWT ... 26
8.3 `@Auth`, et où il est vérifié ... 26
8.4 `Auth::$restrictions` — protégé par défaut ... 27
8.5 Le contrôle de propriété ... 27
8.6 Le profil public ... 28
8.7 Mots de passe, sessions, codes ... 28
8.8 Le reste de la surface ... 29
8.9 Liste de vérification ... 29

Chapitre 9 — La documentation d'API ... 31
9.1 Ce qui est généré ... 31
9.2 Ce qui n'est pas lu ... 31

Chapitre 1 — Les annotations de route

1.1 Le lecteur d'annotations

Devups lit ses annotations de route avec `maslosoft/addendum`, via `ReflectionAnnotatedClass` et `$method->hasAnnotation('X')` / `getAnnotation('X')`. Ce sont des annotations en docblock, pas des attributs PHP 8 :

```
/**
 * @Auth(authorized=1)
 * @GET(path='/:id/comments')
 */
public function comments($id) { ... }
```

Les classes d'annotation vivent dans `dclass/lib/annotation/`.

1.2 Les annotations disponibles

Annotation	Portée	Rôle
`@Api(name='/xxx')`	classe	le préfixe d'URL d'un `FrontController`
`@GET(path='...')`	méthode	route en GET
`@POST(path='...')`	méthode	route en POST
`@PUT(path='...')`	méthode	route en PUT ; `path` vaut `'/:id'` par défaut
`@DELETE(path='...')`	méthode	route en DELETE ; `path` vaut `'/:id'` par défaut
`@Api` (sur une méthode)	méthode	route tous verbes
`@Auth(authorized=1)`	méthode	exige un jeton JWT valide
`@MethodView(...)`	méthode	l'ancienne forme, utilisée dans `route/Admin.php`
`@DDoc(...)`	méthode	documentation
`@OA\...`	classe / méthode	annotations OpenAPI (`zircote/swagger-php`), documentation seulement

`GET`, `POST`, `PUT` et `DELETE` héritent de `Api`. Chacun apparie l'URL réelle au motif `<name de la classe>` + `<path de la méthode>` et verse les paramètres extraits dans `Request::$uri_get_param`. Si l'appariement échoue : 404.

1.3 Les segments de chemin

Un segment préfixé de `:` est un paramètre nommé :

```
/** @GET(path='/:id') */
public function detail($id) { ... }

/** @GET(path='/export/:format/:id') */
public function export($format = null, $id = null) { ... }
```

Le nom du paramètre d'URL doit correspondre au nom du paramètre PHP : c'est par ce nom que la valeur est liée. Un `path` peut porter plusieurs motifs séparés par des virgules ; le premier qui a le bon nombre de segments est retenu.

1.4 La barre oblique initiale

Le motif complet est construit par `rtrim($ctrlApi->name, '/') . '/' . ltrim($ann->path, '/')`. La barre oblique initiale du `path` reste la forme attendue : `@GET(path='/export/:format/:id')`, jamais `@GET(path='export/:format/:id')`.

Chapitre 2 — Les trois styles d'API

C'est la distinction à ne pas confondre. Les trois styles vivent sur les mêmes entités et ne se comportent pas pareil.

2.1 Style générique — `<action>/<entité>`

Piloté par `WebService::systemServe()` et les méthodes `createCore()`, `updateCore()`, `deleteCore()`, `detailCore()`, `uploadCore()`, `ll()`, `dcollection()` de `FrontController.php`.

L'entité doit figurer dans `$public_access` de `route/WebService.php` **et** dans `config/dvups_configurations.php`, sinon :

```
entity <x> has no public access or has not been found
```

Action	Route	Méthode du socle
Créer	`POST /api/create/<entité>`	`createCore()`
Détail	`GET /api/detail/<entité>?id=X`	`detailCore($id)`
Lister	`GET /api/lazyloading/<entité>?dfilters=on&...`	`ll()`
Mettre à jour	`/api/update/<entité>?id=X`	`updateCore($id)`
Supprimer	`/api/delete/<entité>?id=X`	`deleteCore($id)`
Envoyer un fichier	`/api/upload/<entité>?id=X`	`uploadCore($id)`
Lectures groupées	`POST /api/dcollection`	`dcollection()`

> **Sur les verbes HTTP de ce style.** Le dispatcher ne contrôle pas le verbe : il teste seulement le premier segment contre la liste des sept actions. Écrire le verbe correct reste la bonne pratique ; une version future peut resserrer.

La charge utile

```
{ "task": { "name": "valeur", "project.id": 3 } }
```

Le point référence une relation `manyToOne`. Pour une création en masse :

```
{ "task_bulk": [ { ... }, { ... } ] }
```

`createCore()` traite alors chaque élément séparément et renvoie un tableau `datacollection` avec un `success` par ligne.

L'envoi de fichiers

Un attribut de type fichier ne passe pas en JSON : il faut du `multipart/form-data`, avec des clés `<entité>_form[champ]` :

```
cmsresource_form[name]
cmsresource_form[image]      (le fichier)
```

`createCore()` détecte `$_FILES[<entité>_form]` et enchaîne un `upload<Champ>()` après la création.

Deux pièges du listing

1. `?<relation>_id=X` ne filtre rien. Le bon filtre relationnel est :

```
?dfilters=on&project.id:eq=1
```

La première forme répond 200, `success: true`, avec des résultats — ceux de toute la table.

2. La réponse est paginée, silencieusement. Par défaut `per_page = 30`. La réponse porte `nb_element` (le total réel du filtre), `per_page`, `current_page`, `next`, `remain` (1 tant qu'il reste des pages). Les paramètres de pagination sont `?next=<page>&per_page=<taille>` — ni `?page=`, ni `?p=`.

```
let all = [], page = 1, remain = true
while (remain) {
  const res = await api.get(`/lazyloading/task?dfilters=on&project.id:eq=${id}&next=${page}
&per_page=100`)
  all = all.concat(res.data.listEntity || [])
  remain = !!res.data.remain
  page = res.data.next
}
```

2.2 Style REST dédié — un `FrontController` par entité

Généré par `php devups core:g:frontcontroller`. La classe porte `@Api(name='/xxx')`, les méthodes une annotation de verbe.

```
/**
 * @Api(name='/task')
 */
class TaskFrontController extends FrontController
{
    /** @GET */
    public function index()
    {
        return Task::initQb()->with('project', ['name'])->lazyloading();
    }

    /** @GET(path='/:id') */
    public function detail($id) { ... }

    /**
     * @Auth(authorized=1)
     * @POST
     */
    public function create() { ... }

    /**
     * @Auth(authorized=1)
     * @PUT(path='/:id')
     */
    public function update($id) { ... }

    /**
     * @Auth(authorized=1)
     * @DELETE(path='/:id')
     */
    public function delete($id) { ... }

    /**
     * @Auth(authorized=1)
     * @POST(path='/:id/close')
     */
    public function close($id) { ... }
}
```

Méthode PHP	Annotation	Route
`index()`	`@GET`	`GET /api/task`
`create()`	`@POST`	`POST /api/task`
`detail($id)`	`@GET(path='/:id')`	`GET /api/task/42`
`update($id)`	`@PUT(path='/:id')`	`PUT /api/task/42`
`delete($id)`	`@DELETE(path='/:id')`	`DELETE /api/task/42`
`close($id)`	`@POST(path='/:id/close')`	`POST /api/task/42/close`

Ce style est strict sur le verbe : une méthode annotée `@GET` appelée en `POST` répond 405.

Le dispatch

`Request::FrontController()` :

1. convertit le segment d'action en nom de méthode (`close-task` → `closeTask`) ;
2. vérifie que l'entité demandée correspond au `name` de l'annotation `@Api` de la classe (sinon 500) ;

3. si la méthode n'existe pas, balaie les méthodes publiques qui portent une annotation du verbe courant avec un `path`, et apparie le motif complet à l'URI — c'est le mécanisme des sous-ressources ;
4. sinon, résout par nombre de segments (`/task` → `index` / `create` /..., `/task/42` → `detail` / `update` / `delete`) ;
5. contrôle `Auth::$restrictions` sur le nom de méthode réellement résolu (§8.4) ;
6. contrôle `@Auth` ;
7. **verrou d'annotation** : si la méthode n'a ni annotation du verbe courant, ni `@Api`, répond 405 ;
8. appelle la méthode, paramètres liés par nom.

> **Conséquence du verrou (étape 7).** Une méthode publique de `FrontController` sans annotation de verbe n'est atteignable par aucune route, quel que soit son nom. Vérifier l'annotation avant de conclure qu'une méthode est appelée.

2.3 Style libre — `<action>Serve()`

Pour tout ce qui n'est ni une entité ni un CRUD : une méthode `<Action>Serve()` directement sur `route/WebService.php`.

```
// GET /api/hello → helloServe()
public function helloServe() { ... }

// POST /api/accept-invitation?id=5 → acceptInvitationServe()
public function acceptInvitationServe() { ... }
```

Les paramètres passent en chaîne de requête. Le routeur n'exige pas d'annotation ici : `@Auth` est à poser explicitement, ou le contrôle à écrire à la main :

```
if (!Auth::$user_id) {
    http_response_code(401);
    Genesis::json_encode(['success' => false, 'detail' => 'non authentifié']);
}
```

`Auth::$user_id` est déjà peuplé à ce stade : `runServe()` appelle `Auth::execute()` en tête.

2.4 Lequel choisir

Besoin	Style
CRUD immédiat d'une entité, sans écrire de contrôleur	générique
API consommée par une application mobile ou un front séparé, avec un contrat stable	REST dédié
sous-ressource (`/task/:id/close`)	REST dédié, avec `@POST(path='/:id/close')`
action métier qui n'est pas un CRUD	libre (`xxxServe`)

Le style générique est pratique et dangereux : il expose les actions d'écriture de toute entité enregistrée. Le chapitre 8 décrit les garde-fous.

Chapitre 3 — Les contrôleurs

3.1 Les quatre familles

Classe	Rôle	Dispatch
`<Entité>Controller`	le CRUD du back-office	`Request::Controller()`
`<Entité>FrontController`	l'API REST dédiée	`Request::FrontController()`
`Module<Nom>` (extends `ModuleController`)	les écrans d'un module	`Request::Module()`
`route/App`, `WebService`, `Admin`, `AdminService`	les quatre portes	`Router`

3.2 Le contrôleur d'administration

`Request::Controller()` convertit le dernier segment de l'URL en nom de méthode :

URL	Méthode	Action contrôlée par `can()`
`/admin/mp/ModuleTask/task/list`	`listView()`	`read`
`.../task/datatable`	`datatable()`	`read`
`.../task/detail?id=4`	`detailView(4)`	`read`
`.../task/form?id=4`	`formView(4)`	`read`
`.../task/create`	`createAction()`	`create`
`.../task/update?id=4`	`updateAction(4)`	`update`
`.../task/delete?id=4`, `deletegroup`	`deleteAction()`, `deletegroupAction()`	`delete`
`.../task/export`, `import`	`exportAction()`, `importAction()`	`export`, `import`

> **Nouveau en 7.0.** Avant d'appeler la méthode, `Request::Controller()` vérifie `can($action, $entité)`. Refus : 403, `{"success":false,"detail":"forbidden"}`. Cela vaut pour tout contrôleur d'entité, qu'il utilise `CrudTrait` ou ses propres méthodes. Une action métier (`closeAction()`) ne figure pas dans cette table : son contrôleur appelle `can('close', Task::class)` lui-même.

3.3 `CrudTrait` — le CRUD en six méthodes

Un `<Entité>Controller` généré utilise `CrudTrait`, qui fournit :

Méthode	Rôle
`listView($next, $per_page)`	construit la liste par `<Entité>Table::init(...)->buildindextable()` et rend la vue
`datatable($next, $per_page)`	renvoie la liste en JSON, pour le rechargement sans rechargement de page

Méthode	Rôle
`formView($id = null)`	construit le formulaire, avec pour action `admin/api/<entité>/create` ou `update?id=`
`createAction()` / `updateAction($id)`	remplissent l'entité par `form_fillingentity()`, renvoient les erreurs ou la ligne mise à jour
`detailView($id)`	rend la fiche de détail
`deleteAction($id)` / `deletegroupAction($ids)`	suppression simple et en masse

Le contrôleur généré se contente de déclarer quatre noms :

```
class TaskController extends Controller
{
    use CrudTrait;
    public static $classname = 'Task';
    public static $entityname = 'task';
    public static $tablename = 'TaskTable';
    public static $formname  = 'TaskForm';
}
```

3.4 Le remplissage d'une entité depuis une requête

Méthode	Entrée	Usage
`form_fillingentity($object, $data)`	`$_POST[<entité>_form]`	les formulaires du back-office
`hydrateWithJson($object, $jsonform, $deeper = false)`	un tableau décodé depuis le corps JSON	l'API
`form_generat($object, $data, $deeper)`		

Les deux premières alimentent `$this->error`, un tableau indexé par nom de champ. Un contrôleur vérifie `if ($this->error)` avant de persister. Le cœur du remplissage appelle, pour chaque champ, le `set<Champ>()` de l'entité s'il existe, et traite sa valeur de retour comme un message d'erreur (Tome II, §5.4).

3.5 Les écouteurs d'événements

```
Controller::addEventListener($position, $action, $classname);
Controller::addEventListenerAfterCreate($classname, $method);
```

`createCore()` s'en sert pour enchaîner un `upload<Champ>()` après la création, quand des fichiers accompagnent la requête.

3.6 Les contrôleurs de module

Un `Module<Nom>.php` hérite de `ModuleController`, donc de `Router`. `Request::Module()` convertit le segment en `<segment>View`.

- Le constructeur d'un `Module<X>` attend un `$default_url` (une chaîne : `'layout'`), jamais l'objet `Router`.
- La barre latérale génère un lien `.../dashboard` pour chaque module. `ModuleController::dashboardView()` est un repli qui appelle `layoutView()` si elle existe. Un module qui définit sa propre `dashboardView()` l'emporte.

3.7 La dégradation gracieuse

Un écran dont le code n'a jamais été généré ne doit pas produire une erreur fatale nue. Si la classe de contrôleur n'existe pas, ou si l'erreur levée est une « Class … not found », `Request::Controller()` et `Request::Module()` rendent `dclass/devups/Resource/views/not_monitored.blade.php` — un écran « contenu non monitoré ». Seule une classe introuvable est dégradée : un vrai bug continue de planter, sinon le message mentirait.

Chapitre 4 — Le moteur de listes

4.1 Le contrat déclaratif

Une liste se déclare dans une classe `<Entité>Table` qui hérite de `Datatable`, et un moteur unique (`Datatable.php`) rend toutes les listes.

```
class TaskTable extends Datatable
{
    public $entity = "task";

    public static function init(\Task $task = null)
    {
        $dt = new TaskTable($task);
        $dt->entity = $task;
        return $dt;
    }

    public function buildindextable()
    {
        $this->order_by = "this.id desc";
        $this->datatablemodel = [
            ['header' => '#', 'value' => 'id'],
            ['header' => t('Nom'), 'value' => 'name'],
            ['header' => t('Projet'), 'value' => 'Project.name'],
            ['header' => t('Fait'), 'value' => function (\Task $t) {
                return $t->done ? '<span class="badge badge-success">' . t('oui') . '</span>' :
'';
            }],
        ];
        return $this;
    }

    public function builddetailtable()
    {
        $this->datatablemodel = [
            ['label' => t('Nom'), 'value' => 'name'],
            ['label' => t('Description'), 'value' => 'description'],
        ];
        return $this;
    }
}
```

> **Toujours passer l'entité au constructeur.** `new TaskTable($task)`, pas `new TaskTable()` suivi de `$dt->entity = $task`. Le constructeur calcule la classe de l'entité, dont `can()` a besoin pour filtrer les boutons d'action ; sans elle, l'écran lève une `TypeError`.

4.2 Les quatre formes d'une colonne

`Datatable` parcourt `$this->datatablemodel` ; quatre formes sont reconnues, dans cet ordre.

Forme 1 — `'get'` appelable, appelé avec l'entité et `'param'` en second argument s'il est présent.

Forme 2 — pas de `'value'` : la clé est la colonne.

```
'name'       => ['header' => t('Nom')],      // → $entity->name
'project.id' => ['header' => t('Projet')],   // → $entity->project->id
```

C'est la forme que le générateur produit.

> **La clé doit nommer une propriété réelle.** Sinon l'accès échoue dans `ModelTrait::__get()`, qui arrête l'exécution sur un attribut non déclaré : l'écran entier ne s'affiche pas.

Forme 3 — `'value'` appelable, qui reçoit l'entité hydratée. La forme à utiliser pour tout calcul, formatage HTML ou lecture indirecte.

Forme 4 — `'value'` chaîne. `'name'` → `$entity->name` ; une chaîne pointée (`'Project.name'`) est résolue **par accesseurs** (`getProject()` puis `getName()`). D'où la nécessité que les accesseurs existent — c'est la cause la plus fréquente des listes cassées : `class X does not have a method "getY"`.

Deux préfixes spéciaux : `src:<champ>` appelle `show<Champ>()` (fichier ou image), `<Relation>::<collection>` parcourt une collection.

> **Corrigé en 7.0.** Une colonne traduite affichée sans `value` explicite (forme 2) affichait « Array » et levait un avertissement. Elle est désormais résolue dans la langue courante, comme ailleurs dans le moteur.

4.3 Les autres clés d'une entrée

Clé	Rôle
`header`	l'en-tête, pour une liste
`label`	l'en-tête, pour une fiche de détail
`value`, `get`, `param`	la colonne (§4.2)
`search`	active ou désactive le filtre de cette colonne
`order`	contrôle le tri sur cette colonne
`sql`	une expression SQL ajoutée à la projection, aliasée sur la clé
`directive`	des attributs HTML supplémentaires

```
'total' => [
    'header' => t('Total'),
    'sql' => '(SELECT COUNT(*) FROM task t2 WHERE t2.project_id = this.id)',
],
```

4.4 Les propriétés de comportement

Propriété	Défaut	Rôle
`$order_by`	`""`	le tri par défaut, ex. `"this.id desc"`
`$per_page`	`10`	la taille de page
`$enablepagination`	`true`	

Propriété	Défaut	Rôle
`$searchaction`	`true`	la ligne de filtres
`$groupaction`	`true`	sélection multiple
`$isRadio`	`false`	sélection unique
`$enabletopaction` / `$defaulttopaction`	`true`	la barre d'actions du haut, le bouton « ajouter »
`$enablecolumnaction` / `$actionDropdown`	`true`	la colonne d'actions, en menu déroulant
`$mainrowaction`	`"edit"`	l'action principale d'une ligne
`$customactions`, `$topactions`, `$groupactions`	`[]`	actions supplémentaires
`$template`, `$viewdata`		un gabarit de rendu personnalisé
`$trattribut_callback`	`null`	pose des attributs HTML sur une ligne
`$qbcustom`	`null`	un `QueryBuilder` personnalisé

Les boutons d'action par défaut (voir, modifier, supprimer) ne sont affichés que si l'administrateur a l'action correspondante (`can()`).

4.5 Les méthodes utiles

Méthode	Rôle
`buildindextable()` / `builddetailtable()`	à implémenter
`build<X>table()` + `router()`	un modèle alternatif, sélectionné par `?tablemodel=X`
`render()`	la page complète
`getTableRest()`	la liste en JSON
`getSingleRowRest($entity)`	une seule ligne, après création ou mise à jour
`renderentitydata($entity)`	une fiche de détail
`Qb($callback)`	injecte un `QueryBuilder` personnalisé
`addFilterParam($param, $value)`	un filtre figé
`addcustomaction()`, `addgroupaction()`, `setDefaultAction()`, `crud_url()`	actions
`disablepagination()`, `setperpage($n)`, `enablefilter($b)`, `disableColumnAction()`	comportement
`exportCsv()`	export, déclenché par `?dexport=1`

4.6 Le vocabulaire de la requête

Paramètre	Rôle
`dfilters=on`	active le filtrage dynamique

Paramètre	Rôle
`order=<colonne>`, `orderjoin=<relation>`, `dsort`, `dsortjoin`	tri
`next=<page>`, `per_page=<n>`	pagination
`tablemodel=<X>`	sélectionne `build<X>table()`
`dexport=1`	export CSV
`dlang=<iso>`	la langue
`drand`, `dcount`, `dsum`	ordre aléatoire, agrégats
en-tête `Columns`	`with()` sans toucher au code

4.7 Les opérateurs de filtre

Syntaxe : `?dfilters=on&<colonne>:<op>=<valeur>`.

Opérateur	SQL
`eq` / `neq` / `oreq`	`=` / `!=` / `=` en OR
`gt` / `lt`	`>` / `<`
`get` / `let`	`>=` / `<=`
`lk` / `lkr` / `lkl`	`LIKE '%v%'` / `'v%'` / `'%v'`
`in` / `notin`	`IN (...)` / `NOT IN (...)`, valeurs séparées par des virgules
`rein`	`<valeur> IN (<colonne>)`
`empty`	`= ''` si 1, `!= ''` sinon
`isNull` / `notNull`	`IS NULL` / `IS NOT NULL`
`btw`	`BETWEEN a AND b`, bornes jointes par `_` : `created_at:btw=2026-07-01_2026-07-31`

> **`gte` et `lte` n'existent pas.** Un opérateur non reconnu tombe dans le cas par défaut, qui est un `LIKE` : `created_at:gte=X` génère `created_at LIKE '%X%'`, sans avertissement. Toujours `get`, `let` ou `btw`.

Une colonne pointée est résolue comme une relation : `project.id:eq=1`. Le filtrage dynamique ne combine qu'en ET ; pour un OU, `whereGroup()` côté PHP.

4.8 La réponse de `lazyloading`

Clé	Contenu
`success`, `listEntity`	les entités de la page
`nb_element`	le total réel du filtre
`per_page`, `current_page`, `next`, `previous`	pagination
`remain`	1 tant qu'il reste des pages
`aggregations`	si `dcount` / `dsum` ont été demandés

Clé	Contenu
`append_values`	ce que `append($key, $value)` a ajouté

Chapitre 5 — Le moteur de formulaires

5.1 Les trois classes

Classe	Rôle
`FormManager` (abstraite)	la base d'un `<Entité>Form` ; définit le vocabulaire
`FormFactory`	le moteur de rendu : transforme une déclaration de champ en HTML
`Form` (extends `FormFactory`)	une API « à la main », hors contrat déclaratif

5.2 Le contrat déclaratif

```
class TaskForm extends FormManager
{
    public static function init(\Task $task, $action = "")
    {
        return new TaskForm($task, $action);
    }

    public function buildForm()
    {
        $this->title = t('Tâche');
        $this->width = 700;

        $this->fields['name'] = [
            FH_TYPE    => FORMTYPE_TEXT,
            FH_LABEL   => t('Nom'),
            FH_VALUE   => $this->entity->name,
            FH_REQUIRE => true,
        ];
        $this->fields['description'] = [
            FH_TYPE  => FORMTYPE_TEXTAREA,
            FH_LABEL => t('Description'),
            FH_VALUE => $this->entity->description,
        ];
        $this->fields['project.id'] = [
            FH_TYPE    => FORMTYPE_SELECT,
            FH_LABEL   => t('Projet'),
            FH_VALUE   => $this->entity->project->getId(),
            FH_OPTIONS => FormManager::Options_Helper('name', Project::all()),
        ];
        return $this;
    }
}
```

Côté contrôleur (`CrudTrait::formView()`) :

```
TaskForm::init($task, $action)->buildForm()->addDformjs()->renderForm();
```

5.3 Les types de champ

Constante	Valeur	Rendu
`FORMTYPE_TEXT`	`'text'`	`<input type="text">`
`FORMTYPE_EMAIL`, `FORMTYPE_NUMBER`, `FORMTYPE_PASSWORD`		entrées typées
`FORMTYPE_DATE` / `FORMTYPE_DATETIME`		`date` / `datetime-local`
`FORMTYPE_TEXTAREA`	`'textarea'`	`<textarea>`
`FORMTYPE_SELECT` / `FORMTYPE_SELECTGROUP`		`<select>`, avec ou sans `<optgroup>`
`FORMTYPE_CHECKBOX` / `FORMTYPE_RADIO`		cases, boutons radio
`FORMTYPE_FILE` / `FORMTYPE_FILEMULTIPLE`		envoi d'un ou plusieurs fichiers
`FORMTYPE_INJECTION`	`'injection'`	injecte un bloc HTML
`FORMTYPE_INJECT_COLLECTIONFORM`	`'injectioncollection'`	déclaré, **non implémenté** — à ne pas utiliser

Sous-types de fichier : `FILETYPE_IMAGE`, `FILETYPE_DOCUMENT`, `FILETYPE_VIDEO`, `FILETYPE_AUDIO`.

> **Aucun type n'est un répéteur.** Il n'existe pas de champ « liste de valeurs modifiable ». La forme usuelle est un `FORMTYPE_TEXT` avec une consigne de séparation par virgules, validé dans le setter.

5.4 Les clés d'un champ

Constante	Valeur	Rôle
`FH_TYPE`	`'type'`	obligatoire : un `FORMTYPE_*`
`FH_LABEL`, `FH_VALUE`		libellé, valeur courante
`FH_OPTIONS`	`'options'`	choix d'un `select`, `radio` ou `checkbox`
`FH_DIRECTIVE`, `FH_ID`		attributs HTML, identifiant
`FH_FILETYPE`	`'filetype'`	`FILETYPE_*` pour un fichier
`FH_REQUIRE`	`'require'`	marque le champ obligatoire
`FH_IMBRICATE`	`'imbricate'`	formulaire imbriqué

`FormFactory` lit en plus, directement : `values`, `src` (aperçu d'un fichier), `lang` (champ traduit, rendu une fois par langue), `placeholder`, `setter`, `hidden`, `callback`.

5.5 Les aides à la construction des options

Méthode	Rôle
`FormManager::Options_Helper($value, $entitylist, $key = "id")`	`[id => libellé]` depuis une liste d'entités ; `$value` peut être pointé

Méthode	Rôle
`FormManager::Options_ToCollect_Helper(...)`	les options moins celles déjà sélectionnées
`FormManager::key_as_value(array $array)`	`[valeur => valeur]`, pour une énumération

```
FH_OPTIONS => FormManager::Options_Helper('name', Project::all())
FH_OPTIONS => FormManager::key_as_value(['brouillon', 'publié', 'archivé'])
```

`Options_Helper()` résout un libellé issu d'une colonne traduite dans la langue courante (corrigé en 7.0 : il affichait du HTML échappé).

5.6 Les méthodes de `FormManager`

Méthode	Rôle
`buildForm()`	à implémenter
`addField($name, $option)`	équivalent de `$this->fields[$name] = $option`
`addJs($file)` / `addCss($file)`	ressources
`addDformjs()`	charge `dform.js`, requis pour la soumission sans rechargement
`renderForm()`	rend le HTML

Propriétés : `$title`, `$width` (600 par défaut), `$entity`, `$fields`, `$formaction`, `$formbutton`.

5.7 L'API « à la main »

```
Form::open($entity, ["action" => "#", "method" => "post"]);
Form::input('name', $entity->name, ['class' => 'form-control']);
Form::textarea('description', $entity->description);
Form::select('project', $options, $entity->project->getId());
Form::file('image', $entity->image, [], FILETYPE_IMAGE);
Form::submit(t('Enregistrer'));
Form::close();
```

Autres méthodes : `number()`, `email()`, `password()`, `selectGroup()`, `autocomplete()`, `radio()`, `checkbox()`, `filepreview()`, `inputarray()`, `imbricate()` / `closeimbricate()`, `reset()`.

5.8 Un champ traduit

Si l'entité est traduite et que le champ porte `'lang' => true`, `FormFactory` rend un champ par langue. Une fiche d'une entité traduite sans aucune ligne de traduction se charge vide (défaut D15, Tome II §7.4) ; l'enregistrement, lui, refuse avec un motif clair.

Chapitre 6 — Les vues

6.1 Blade

Les vues sont des gabarits Blade (`*.blade.php`) compilés dans `cache/views`. Les chemins de recherche sont dans le tableau global `$viewdir` :

Contexte	Chemins
application publique	`web/views`
back-office	`admin/views`
écran d'entité ou de module	plus `src/<composant>/<Module>/Resource/views`

6.2 `Genesis` — le rendu

Méthode	Rôle
`Genesis::render($view, $data)`	rend une vue
`Genesis::renderView($view, $data)`	rend une vue avec `$viewdir` ; ajoute `moduledata`
`Genesis::getView($view, $data)`	retourne la chaîne au lieu de l'émettre
`Genesis::json_encode($value)`	émet du JSON **et termine la requête**
`Genesis::json_encode2(...)`	la même chose sans terminer
`Genesis::top_action($action)`	la barre d'actions d'une liste, selon les droits

> **`Genesis::json_encode()` termine la requête.** Tout ce qui doit s'exécuter doit l'être avant l'appel. Inversement, un refus émis avec `json_encode2()` laisse l'exécution continuer : après un refus, toujours s'assurer que la suite ne s'exécute pas.

6.3 Les fonctions globales utiles

Déclarées dans `dclass/dvups_functions.php` :

Fonction	Rôle
`route($path, $id, $title)` / `d_url(...)`	une URL avec le préfixe de langue (sauf en administration)
`path($src)`, `d_assets($src)`	`__env . $src`, `__env . 'web/assets/' . $src`
`services($service)`	`__env . 'admin/api/' . $service`
`redirect($url)`	redirection puis arrêt
`getadmin()` / `setadmin($id)`	l'administrateur en session
`can($action, $entité)`	le droit de l'administrateur courant (Tome IV, chapitre 2)

Fonction	Rôle
`local()` / `setlang($iso)`	la langue courante (chapitre 7)
`dv_hash_password()` / `dv_verify_password()`	hachage des mots de passe (§8.7)
`url_format()`, `remove_accents()`, `clean()`, `get_string_between()`	chaînes
`qb_sanitize($value)`	entoure de quotes **sans échapper** — voir Tome II §4.2
`dv_dump(...$args)`	`dump()` puis `die(1)` — un arrêt, pas un journal

6.4 Le JavaScript du socle

`dclass/devupsjs/`, servi par la constante `CLASSJS` :

Fichier	Rôle
`devups.js`	le socle
`model.js`	le modèle côté client : création, édition, soumission
`ddatatable.js`, `DdatatableClass.js`	la liste : rechargement, tri, filtres, pagination
`dform.js`	la soumission d'un formulaire sans rechargement
`dialogbox.js`	les fenêtres modales
`Request.js`, `rest.js`	les appels HTTP (stockage local sous la clé `devups_store`)

Une liste demande `devups.js`, `model.js` et `ddatatable.js` ; un formulaire demande en plus `dform.js` — c'est ce que fait `addDformjs()`.

Chapitre 7 — Les langues de l'interface

> **Nouveau en 7.0.** Les versions 4.x stockaient les traductions de l'interface en base (`Local_content`, `Local_content_key`, `Page_mapped`) et proposaient plusieurs fonctions concurrentes (`langt()`…). Tout cela a été retiré : les traductions vivent dans des fichiers, `t()` ne fait aucune requête SQL, et le code ISO à deux lettres est la seule référence de langue.

7.1 Le code ISO à deux lettres, partout

La langue est identifiée par son code ISO 639-1 (`fr`, `en`…) en session (`local()`), dans l'URL (`?lang=` ou préfixe `/fr/`), en base (`dvups_lang.iso_code`) et dans le nom des fichiers. `header.php` compile une seule fois la correspondance code ↔ id des langues **actives** dans `cache/lang/active.php`, invalidé automatiquement à chaque création, modification ou suppression d'une langue depuis le back-office.

`dv_valid_lang($iso)` n'accepte qu'une valeur de la forme `^[a-z]{2}$` **et** correspondant à une langue active ; sinon elle renvoie `null` et l'appelant retombe sur la langue par défaut. `local()`, `setlang()`, `lang.php` et `t_lang()` passent tous par elle (§8.8).

7.2 Les fichiers de traduction

```
<?php
// lang/fr.php
return [
    'Enregistrer' => 'Enregistrer',
    'Bonjour :nom' => 'Bonjour :nom',
    'Projet introuvable' => 'Projet introuvable',
];
```

```
<?php
// lang/en.php
return [
    'Enregistrer' => 'Save',
    'Bonjour :nom' => 'Hello :name',
    'Projet introuvable' => 'Project not found',
];
```

Le dossier est `LANG_DIR` (`lang/` par défaut). `lang.php`, à la racine, charge le fichier de la langue courante **une seule fois par requête**.

La clé est le texte tel qu'il est écrit dans le code. Chaque fichier contient la version de sa langue.

7.3 Les fonctions

Fonction	Rôle
`t($clé, $params = [])`	la traduction dans la langue courante, ou la clé elle-même si elle manque ; substitue `:param`

Fonction	Rôle
`tt($clé)`	alias de `t()`, utilisé dans les vues
`@tt('clé')`	directive Blade équivalente
`t_lang($clé, $iso, $params = [])`	la traduction dans une langue **explicite**, indépendante de la session
`local()`	la langue courante : `?lang=`, puis la session, puis `__lang`
`setlang($iso)`	fixe la langue en session

```
echo t('Bonjour :nom', ['nom' => $user->firstname]);

// Courriel envoyé à un destinataire dont la langue diffère de celle de l'expéditeur :
$subject = t_lang('Votre compte est activé', $user->lang ?? __default_lang);
```

`t_lang()` est faite pour les courriels (`web/views/mails/*.blade.php`) : la langue du destinataire n'est pas celle de la requête en cours.

7.4 La collecte des clés manquantes

Quand `LANG_COLLECT` est actif — par défaut en développement, **jamais en production** —, une clé absente rencontrée par `t()` est ajoutée à tous les fichiers `lang/*.php`, avec pour valeur la clé elle-même. L'écriture est sûre : verrou sur un fichier dédié, fichier temporaire, puis renommage. Le traducteur n'a plus qu'à remplir les valeurs.

7.5 Les commandes

```
php devups lang:scan      # parcourt le code et complète les fichiers en une fois
php devups lang:missing   # liste, par langue, les clés encore non traduites
```

`lang:scan` analyse le code avec le tokenizer de PHP (`token_get_all()`) : il reconnaît les appels `t('...')` et `tt('...')` dont la clé est littérale, y compris avec des apostrophes échappées. Une clé construite (`t('statut.' . $x)`) est ignorée et signalée.

> **Règle de travail.** Écrire des clés littérales. Une clé construite échappe au scan et à la relecture ; préférer `t('Statut : :s', ['s' => $x])` à `t('statut.' . $x)`.

Chapitre 8 — La sécurité

8.1 Les deux systèmes d'identité

Devups a deux mécanismes d'authentification indépendants.

	API publique	Back-office
Support	jeton JWT	session PHP
Porté par	en-tête `Authorization: Bearer <jwt>` ou `Auth: Bearer <jwt>`	`$_SESSION[ADMIN]`
Identité	`Auth::$user_id` (un `User`)	`getadmin()` (un `Dvups_admin`)
Autorisation	`Auth::$restrictions`, `@Auth`, propriété	`can()`

Dans une requête d'administration, `Auth::$user_id` est toujours vide. C'est pourquoi `Router::$admin_route` existe (§8.5).

8.2 Le JWT

Élément	Valeur
Algorithme	HS512
Secret	`JWT_SECRET`
Charge utile	`iss`, `userId`, plus `iat` / `nbf` / `exp` si `JWT_EXPIRE` est défini

```
$jwt   = Auth::getJWT($user);            // à la connexion
$token = Auth::decodeJWTOrFail($jwt);    // lève sur tout jeton absent, invalide ou expiré
```

> **`JWT_EXPIRE=false` désactive toute expiration** : seule la signature est vérifiée, un jeton émis est valable indéfiniment. Fixer une durée (`+72 hours`) en production. Pour tout code neuf qui décode un jeton hors du flux par annotation, utiliser `decodeJWTOrFail()` plutôt que `decodeJWT()`, dont l'échec silencieux peut être oublié.

8.3 `@Auth`, et où il est vérifié

Dispatcher	Traitement
`Router::mapFunction()` (pages, style libre)	si la méthode porte `@Auth`, exige `Auth::$user_id`
`Request::FrontController()` (REST dédié)	si la méthode porte `@Auth`, vérifie le jeton
`FrontController::authorization()` (générique)	consulte `Auth::$restrictions`

L'absence de `@Auth` sur une méthode `xxxServe()` signifie **aucun contrôle** : la route est anonyme.

8.4 `Auth::$restrictions` — protégé par défaut

C'est le mécanisme central, et il fonctionne par **liste d'exceptions** : toute entité de `$public_access` est protégée sur toutes ses méthodes, sauf celles nommées dans `$publicMethods`.

```
// route/WebService.php
public $public_access = [
    'task' => \Task::class,
    'user' => \User::class,
];

public static $publicMethods = [
    // Lectures anonymes. Protégées : create, update, delete, close.
    'task' => ['index', 'detail'],
    'user' => ['registration', 'authentification', 'tokenize',
               'initresetpassword', 'resetpassword', 'googleAuth', 'appleAuth', 'detail'],
];
```

Le nom comparé est celui de la méthode **réellement résolue** : `index` pour `GET /api/task`, `detail` pour `GET /api/task/42`, le nom de l'action pour une route annotée. Une méthode absente de la liste répond 401 sans jeton.

> **Comment construire `$publicMethods` .** Y mettre ce qu'un visiteur non connecté doit pouvoir atteindre, et rien de ce qui touche au compte, à l'écriture ou à l'argent. Nommer en commentaire les actions laissées protégées, pour qu'on ne les prenne pas pour des oublis : la liste porte sa propre justification.

8.5 Le contrôle de propriété

`Auth::$restrictions` exige un jeton, pas que la ligne appartienne au porteur du jeton. Un second mécanisme s'en charge, dans `FrontController`, configurable par projet :

```
// au démarrage de l'application (route/WebService.php, par exemple)
FrontController::setOwnerColumns([
    'task'    => 'user_id',
    'comment' => 'author_id',
]);
```

Pour une entité de cette liste, `updateCore()`, `deleteCore()` et `uploadCore()` vérifient, par une requête dédiée, que la ligne a bien `<colonne> = Auth::$user_id` ; sinon 403.

Trois décisions de conception :

1. **L'absence d'une entité est un choix.** Elle reste protégée par le jeton, sans contrôle de propriété supplémentaire.
2. **Le contrôle passe par une requête dédiée** (`where(...)->count()`), pas par la propriété de l'entité hydratée, qui n'expose pas toujours la colonne de clé étrangère.
3. **`Router::$admin_route` court-circuite le contrôle.** Ces méthodes sont partagées avec l'API du back-office, où `Auth::$user_id` est vide et où un administrateur ne doit évidemment pas être restreint à « ses » lignes.

8.6 Le profil public

`FrontController::restrictForRequester($entity, $id)` appelle `restrictToPublicProfile()` sur l'entité quand le demandeur n'en est pas le propriétaire (et hors back-office). `User` l'implémente : `GET /api/user/42` renvoie le profil public à un tiers, sans e-mail ni téléphone, et le profil complet au titulaire du jeton. `detailCore()` et le listing générique l'appliquent.

8.7 Mots de passe, sessions, codes

> **Nouveau en 7.0.** Tout ce paragraphe.

Mots de passe. `Dvups_admin` et `User` stockent `password_hash()` (`dv_hash_password()`), vérifié par `password_verify()` (`dv_verify_password()`). Un ancien hash `sha1()` encore en base est accepté une dernière fois, puis remplacé automatiquement au premier login réussi : aucune réinitialisation forcée.

« Se souvenir de moi » (back-office). Le cookie ne contient jamais le mot de passe ni son hash : il contient un jeton aléatoire dont seul le hash est stocké en base, avec une date d'expiration. Le jeton est renouvelé quand il sert à restaurer une session, et invalidé à la déconnexion comme au changement de mot de passe. Cookies `HttpOnly`, `SameSite=Lax`, et `Secure` en HTTPS.

Réinitialisation du mot de passe (API). `POST /api/user/initresetpassword` émet un jeton dédié (`random_bytes(32)`, encodé en base64url), dont seul le `sha256` est stocké, valable `RESET_PASSWORD_TOKEN_TTL_MINUTES` et à usage unique. Le lien envoyé par courriel ne contient ni l'identifiant de l'utilisateur ni le code court. La réponse est identique que l'adresse existe ou non : pas d'énumération de comptes. `POST /api/user/resetpassword` consomme le jeton.

Codes à 6 chiffres (activation de compte, changement de téléphone). Ils expirent après `ACTIVATION_CODE_TTL_MINUTES` et sont invalidés après `RATE_LIMIT_CODE_MAX_ATTEMPTS` essais erronés.

Limitation de fréquence. `RateLimiter` (`dclass/devups/RateLimiter.php`) compte les tentatives par clé dans une fenêtre glissante (stockage fichier, `cache/ratelimit/`) :

```
$limiter = new RateLimiter('login_admin:' . strtolower($login),
                           RATE_LIMIT_LOGIN_WINDOW_SECONDS, RATE_LIMIT_LOGIN_MAX_ATTEMPTS);
if ($limiter->tooManyAttempts()) {
    http_response_code(429);
    return ['success' => false, 'detail' => t('Trop de tentatives, réessayez plus tard.'),
            'retry_after' => $limiter->retryAfter()];
}
// … en cas d'échec :  $limiter->hit();   en cas de succès :  $limiter->clear();
```

Il protège la connexion au back-office et à l'API (par compte et par IP), la vérification et le renvoi des codes.

L'adresse IP du client. `Util::get_IP_address()` ne fait confiance à `X-Forwarded-For` (et aux en-têtes assimilés) que si la connexion vient d'une adresse listée dans `TRUSTED_PROXIES`. Sans cette liste, l'en-tête est ignoré : un client ne peut pas se forger une IP pour contourner la limitation. Derrière un reverse proxy ou un équilibreur de charge, renseigner son adresse, sinon tous les clients partageront l'IP du proxy.

8.8 Le reste de la surface

Les droits du back-office. `can($action, $entité)`, vérifié par `Request::Controller()` pour toute action CRUD, par les boutons des listes et par les contrôleurs d'administration eux-mêmes (Tome IV, chapitre 2).

Les points d'entrée de module. `services.php` exige une session d'administration, ainsi que l'`index.php` des composants `devups` et `devupscms` (Tome I, §2.4).

La racine du site. Le `.htaccess` racine refuse l'accès direct aux fichiers cachés (dont `.env`), à `config/`, `database/`, `cache/`, `vendor/`, `tests/`, et au code PHP de `src/` et `dclass/` hors des points d'entrée prévus.

Les tâches planifiées. `cron.php` ne s'exécute qu'en ligne de commande, ou par HTTP avec `?token=<CRON_TOKEN>`, comparé en temps constant.

La langue. Une valeur de `?lang=` n'entre jamais dans un chemin de fichier sans passer par `dv_valid_lang()` (§7.1), puis par un contrôle `realpath()` qui vérifie que le fichier est bien dans `LANG_DIR`. Une valeur invalide retombe sur la langue par défaut, sans erreur.

Le chemin générique et les entités sensibles. `WebService` propose deux listes, vides par défaut :

Liste	Effet
`$genericPathMoneyEntities`	le chemin générique ne sert pas du tout ces entités, lectures comprises : elles passent par leur `FrontController` dédié
`$genericPathWriteBlockedEntities`	le chemin générique ne sert que leurs lectures

Le refus répond 403 sans dire si la ligne existe, et **termine la requête** : un refus suivi de la poursuite de l'exécution a déjà existé, et l'écriture se faisait quand même.

CORS. Un en-tête personnalisé nouveau doit être ajouté à la liste codée en dur dans `Router.php` (Tome I, §5.4), sinon le navigateur refuse la requête au vol préliminaire.

8.9 Liste de vérification

Point	À vérifier
nouvelle entité exposée	dans `$public_access` ? dans `$publicMethods` à bon escient ?
nouvelle action d'écriture d'API	`@Auth(authorized=1)` ? absente de `$publicMethods` ?
écriture sur une ressource d'un utilisateur	l'entité est-elle dans `setOwnerColumns()` ? sinon, est-ce un choix documenté ?
nouvelle action d'administration	`can()` vérifié — automatiquement pour le CRUD, à la main pour une action métier ? `rights:sync` lancé ?
nouveau refus	la requête s'arrête-t-elle vraiment ? le message révèle-t-il l'existence de la ressource ?
entrée utilisateur dans un chemin de fichier	validée par liste blanche, puis `basename()` ou `realpath()` ?
valeur utilisateur dans `whereIn()` / `whereBetween()`	castée ou validée ?

Point	À vérifier
nouvel en-tête HTTP	ajouté à `Access-Control-Allow-Headers` ?
méthode de `FrontController`	porte-t-elle une annotation de verbe ? sinon elle est morte
secret	dans `.env`, jamais dans un fichier versionné
production	`APP_PROD=true`, `JWT_EXPIRE` défini, `TRUSTED_PROXIES` renseigné derrière un proxy

Chapitre 9 — La documentation d'API

> **Nouveau en 7.0.**

9.1 Ce qui est généré

La documentation est construite par analyse statique des annotations Devups et des datacores, sans charger aucune classe PHP (`dclass/generator/OpenApiGenerator.php` et `DocGuideGenerator.php`).

Accès	Contenu	Protection
`/documentation/`	spécification OpenAPI des routes front (JWT), avec Swagger UI	publique
`/documentation/guide.php`	guide développeur interactif, depuis les datacores (`?format=json` pour le JSON brut)	publique
`/admin/apidocs`	Swagger UI complet : routes front **et** routes du back-office (session)	`can('read', 'Documentation')`
`php devups api:doc`	écrit `openapi.json` à la racine (routes front)	fichier non versionné
`php devups doc:guide`	écrit une page HTML par entité sous `documentation/guide/<composant>/<module>/<entité>.html`	fichiers non versionnés

`Documentation` est une action « virtuelle » : `rights:sync` la crée comme n'importe quelle action d'entité, ce qui permet de donner accès au Swagger du back-office à un rôle autre que `admin`.

> **`openapi.json` dans la chaîne d'intégration.** `php devups api:doc` en CI produit un fichier que l'on peut comparer d'une version à l'autre, ou donner à un générateur de client.

9.2 Ce qui n'est pas lu

Le générateur déduit chaque route de ses annotations Devups (`@Api`, `@GET`, `@POST`, `@PUT`, `@DELETE`, `@Auth`) et chaque schéma du datacore de l'entité. Les annotations `@OA\...` de `zircote/swagger-php`, que l'on trouve sur certains contrôleurs, ne sont **pas** lues par ce générateur : elles ne changent rien à `/documentation/` ni à `/admin/apidocs`. Une route qui n'apparaît pas dans la documentation est d'abord une route dont l'annotation de verbe manque — ce qui, pour un `FrontController`, la rend aussi inatteignable (§2.2).

Fin du Tome III. Le Tome IV traite des modules fournis par le socle : droits, journaux, langues, contenus, arbres, statuts, notifications, courriels, et des dettes connues du framework.